



المعهد الوطني للبريد والمواصلات
المعهد الوطني للبريد والمواصلات
Institut National des Postes et Télécommunications



SMART
AUTOMATION
TECHNOLOGIES

Projet d'été – Cybersécurité Industrielle

Domaine : Détection d'Intrusions dans les Systèmes Industriels (ICS/SCADA)

Rapport de Stage

Conception et déploiement d'un système intelligent de
détection des cyberattaques temps-réel affectant les
automates (PLC) d'une mini-ligne de production connectée

Encadrants :

Mr. ...

Mr. ...

Membres du projet :

...

Année universitaire 2025–2026

Table des matières

Liste des Abréviations	3
1 Introduction et Contexte du Projet	4
1.1 Contexte Général	4
1.2 Architecture du Système de Production	4
1.3 Simulation de la Ligne de Production	6
2 Implémentation Technique	8
2.1 Environnement de Développement	8
2.2 Génération du Trafic Industriel	8
2.2.1 Génération Modbus TCP	8
2.2.2 Génération de datasets S7comm classique	10
2.2.3 Génération S7comm Plus (S7+ Sécurisé)	11
2.3 Prétraitement et Extraction des Données	13
2.4 Entraînement des Modèles	15
2.5 Évaluation Cross-Dataset	18
2.6 Résultats et Analyse	19
2.6.1 Détection par Protocole	20
3 Déploiement et Validation en Environnement Industriel	22
3.1 Architecture de Test	22
3.2 OpenPLC Runtime	22
3.3 Interface de Supervision Node-RED	23
3.3.1 Composants Node-RED	24
3.4 Pipeline de Détection en Temps-Réel	26
3.4.1 Processus Producteur (Sniffer Redis)	27
3.4.2 Processus Consommateur (Détection ML)	28
3.4.3 Exemple d'Exécution	28
3.4.4 Performances du Pipeline	29
3.4.5 Intégration des Notifications	29
3.5 Conclusion du Chapitre	30

Liste des Abréviations

Abréviation	Signification
IoT	Internet of Things
ICS	Industrial Control System
SCADA	Supervisory Control and Data Acquisition
PLC	Programmable Logic Controller
HMI	Human-Machine Interface
OT - IT	Operational Technology - Information Technology
IDS/IPS	Intrusion Detection/Prevention System
ML/DL	Machine/Deep Learning
Modèles et Algorithmes	
XGBoost	Extreme Gradient Boosting
RNN	Recurrent Neural Network
LSTM - AE	Long Short-Term Memory - Autoencoder
SVM	Support Vector Machine
ISO	Isolation Forest
SMOTE	Synthetic Minority Over-sampling Technique
Couches et Protocoles Réseau	
MBAP	Modbus Application Protocol
TLV	Type-Length-Value
TPKT	Transport Protocol Data Unit (RFC 1006)
COTP	Connection-Oriented Transport Protocol (ISO 8073)
PDU	Protocol Data Unit (Unité de Données de Protocole)
ROSCTR	Record-Oriented Security Control (S7comm PDU Type)
TIA Portal	Totally Integrated Automation Portal (Siemens)
Métriques	
TP / TN / FP / FN	True Positives / True Negatives / False Positives / False Negatives
FPR	False Positive Rate (Taux de Faux Positifs)
MSE	Mean Squared Error
ROC	Receiver Operating Characteristic
AUC	Area Under Curve
IAT	Inter-Arrival Time
BPF	Berkeley Packet Filter

Table 1 – Liste des abréviations utilisées dans le projet

1 Introduction et Contexte du Projet

1.1 Contexte Général

Les systèmes industriels critiques (ICS, SCADA, PLC) constituent le cœur des infrastructures modernes dans les domaines de l'énergie, du transport, de la fabrication et de la distribution d'eau. Ces systèmes contrôlent des processus physiques essentiels où une défaillance peut avoir des conséquences catastrophiques.

Avec la convergence de l'OT et de l'IT, ces environnements deviennent de plus en plus connectés et exposés aux cyberattaques. Contrairement aux systèmes informatiques classiques, les PLC industriels présentent des contraintes spécifiques :

- Disponibilité et intégrité prioritaires sur la confidentialité
- Protocoles propriétaires ou industriels (Modbus, S7, DNP3)
- Ressources limitées (CPU, mémoire, bande passante)
- Cycle de vie long (15-20 ans)
- Conséquences physiques potentielles des attaques

Ce projet vise à concevoir et déployer un système intelligent de détection d'intrusions temps-réel pour les PLC d'une mini-ligne de production connectée.

1.2 Architecture du Système de Production

Les lignes de production connectées se basent principalement sur la communication en protocoles industriels entre les outils de productions (automates, serveurs) en touche avec les composants logiciels. Les protocoles qui nous intéressent sont :

- **Modbus TCP** : Protocole de communication industriel standardisé (port 502) utilisé pour échanger des données entre le superviseur (SCADA/HMI) et les équipements de terrain (PLC, capteurs, actionneurs). Il fonctionne en mode client/serveur sur TCP/IP.
- **S7comm** : Protocole propriétaire Siemens (port TCP 102) pour la communication entre PLC S7, HMI et SCADA. Permet la lecture/écriture de variables et le diagnostic. Vulnérable car ni authentification ni chiffrement.
- **S7comm Plus** : Version sécurisée de S7comm (S7-1200 V4.0+, S7-1500) intégrant chiffrement TLS, authentification mutuelle et protection d'intégrité. Utilise des clés éphémères contre le replay.

Ainsi, la structure globale de projet :

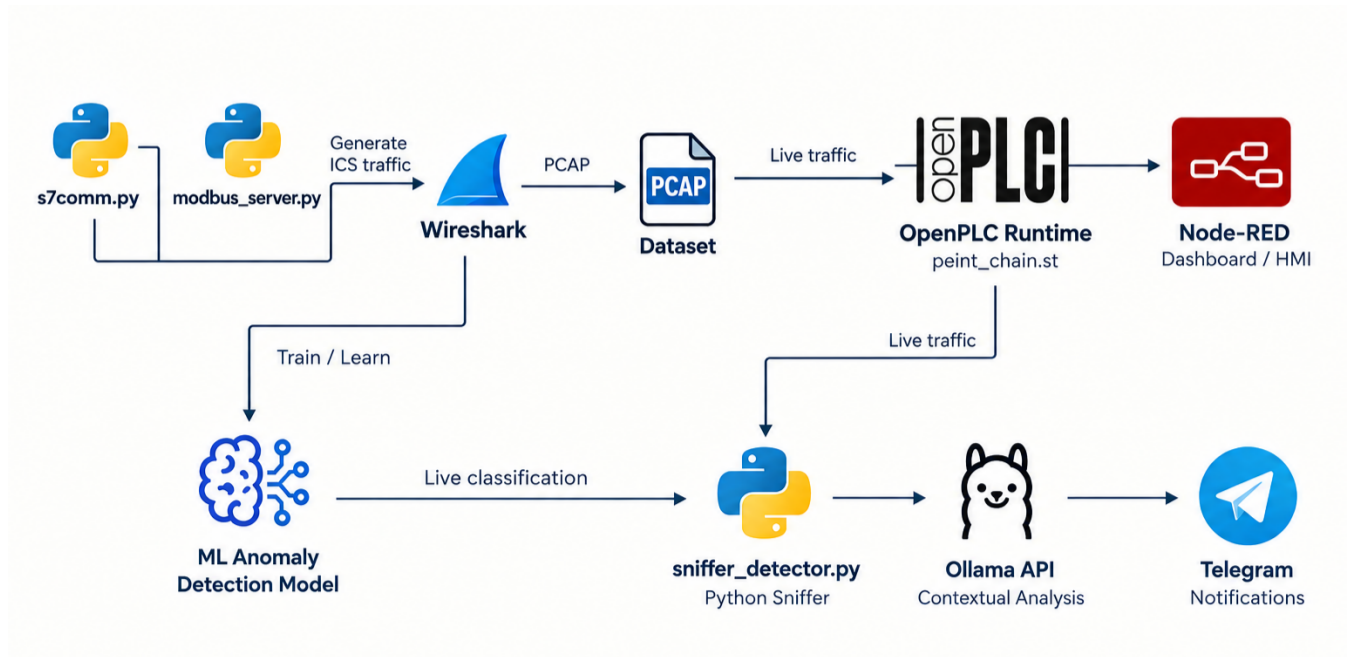


Figure 1.1 – Architecture du système industriel de production

Les cinq couches du système sont :

1. **Couche de Génération de Trafic** : Simulation des protocoles industriels (Modbus TCP, S7comm, S7comm Plus)
2. **Couche de Capture & Ingestion** : Acquisition des paquets réseau via Wireshark/tshark
3. **Couche d'Extraction de Caractéristiques** : Parsing des PCAP et génération du dataset
4. **Couche d'Apprentissage Automatique** : Entraînement, validation des modèles IDS et analyse approfondu.
5. **Couche d'Inférence & Alerting** : Détection temps-réel et notifications

Composants du système de production

- **OpenPLC Runtime** : Automate programmable industriel (PLC) open source qui exécute la logique de contrôle en temps réel. Il reçoit des instructions via Modbus TCP et actionne les sorties physiques (moteurs, vannes, capteurs) selon le programme ladder chargé. Effectivement il accepte deux formes principales :
 - LD (Ladder Diagram / Schéma de contact) : Représentation graphique inspirée des circuits électriques à relais, très populaire en automatisme.
 - ST (Structured Text / Texte structuré) : Langage textuel de haut niveau, similaire au Pascal ou au C, idéal pour les algorithmes complexes et les calculs mathématiques. <Voir> peint-chain.st </voir>
- **Node-RED** : Plateforme de programmation visuelle "flow-based" riche en modules qui sert d'interface SCADA et HMI. Elle permet de créer des dashboards de supervision, de visualiser

les données des capteurs en temps réel, et d'envoyer des commandes aux PLC via des nœuds Modbus.

- **Python PyModbus** : Bibliothèque Python qui implémente le client Modbus. Elle est utilisée pour injecter du trafic Modbus (normal ou malveillant) à des fins de simulation, de test et de génération de données d'entraînement pour les modèles ML.
- **Snap7** : Bibliothèque open source implémentant les protocoles S7comm et S7comm Plus. Permet la simulation de PLC Siemens, l'injection de trafic et la génération de données d'entraînement sans matériel physique.
- **Wireshark** : Outil d'analyse réseau qui capture et analyse les trames Modbus TCP en mode promiscuité. Il permet d'inspecter le trafic en profondeur, de visualiser les échanges entre les équipements, et de collecter les métriques nécessaires à la détection d'anomalies avec les filtres adéquats.
- **Bettercap** : Framework puissant pour les attaques réseau, le spoofing et le MITM (Man-in-the-Middle). Il permet d'intercepter, modifier et réinjecter des trames Modbus TCP et S7comm, simulant des attaques réalistes pour évaluer l'efficacité du système de détection.
- **Nmap** : Scanner réseau polyvalent utilisé pour découvrir les équipements industriels, identifier les ports ouverts (notamment 502 et 102) et les services exécutés. Il permet également la détection de vulnérabilités via ses scripts NSE(Nmap Scripting Engine).
- **Metasploit** : Framework d'exploitation de vulnérabilités contenant des modules spécifiques aux systèmes industriels. Il permet de simuler des attaques avancées contre les PLC, d'exécuter des exploits et de tester les capacités de détection face à des menaces complexes.

1.3 Simulation de la Ligne de Production

La ligne de production simulée représente un processus industriel de peinture des bouteilles et contrôle de qualité, organisé en quatre phases distinctes :

Phase 1 : Convoyeur d'Alimentation

Un convoyeur à bande motorisé alimente le système en pièces brutes. Un capteur photoélectrique détecte l'arrivée des pièces et déclenche le processus.

Phase 2 : Station de Peinture

La pièce est immobilisée par un vérin pneumatique sous une buse de peinture. Le cycle de pulvérisation est déclenché, recouvrant la surface de la pièce en toutes les directions.

Phase 3 : Contrôle Qualité

Un capteur optique inspecte la qualité de la peinture dans la couverture et l'épaisseur ainsi la coloration. Une tour lumineuse (vert/rouge) indique le résultat du contrôle.

Phase 4 : Éjection et Tri

Un poussoir pneumatique sépare les pièces : les conformes continuent, les défectueuses sont rejetées sur une goulotte latérale.



Figure 1.2 – *Layout de la ligne de production simulée*

2 Implémentation Technique

2.1 Environnement de Développement

Composant	Spécification
Système d'exploitation	Windows 11 / Ubuntu 22.04 LTS
Langage	Python 3.13 - Structured Text
Bibliothèques réseau	Scapy, PyShark, python-snap7, PyModbus
Bibliothèques ML	scikit-learn, XGBoost, PyTorch
Base de données	Redis (buffer temps-réel en cache)
Visualisation	Node-RED (customised dashboard)
LLM Local	Ollama API (llama3.2 :3b)
Notifications	Telegram Bot API

Table 2.1 – *Environnement technique du projet*

2.2 Génération du Trafic Industriel

2.2.1 Génération Modbus TCP

Le trafic Modbus TCP est généré via un serveur PyModbus dans le script `modbus_server.py` qui implémente :

- Un serveur Modbus TCP sur le port 502
- Une simulation physique avec bruit gaussien et variations sinusoïdales dans les valeurs de température et pression , comme aussi les probabilités aléatoires pour les pièces défectueuses.
- Des cycles de production complets (démarrage, peinture, contrôle qualité, éjection) avec temporisation adéquate.
- Une table d'adressage Modbus définie comme suit :

Adresse	Nom	Description
— Sorties PLC (Actionneurs) —		
0	COIL_CONVEYOR	M_Convoyeur : Commande du moteur du convoyeur. Active (1) = convoyeur en marche, pièce en déplacement vers le poste de peinture.
1	COIL_PAINTING	V_Peinture : Vanne de peinture. Active (1) = pulvérisation de peinture en cours sur la pièce.
2	COIL_EJECT	V_Eject : Vanne d'éjection. Active (1) = pièce défectueuse rejetée vers le bac de rebut.
— Entrées HMI (Commandes Opérateur) —		
8	COIL_START	HMI_Start : Commande de démarrage du cycle. L'opérateur appuie sur "Démarrer" pour lancer la production.
9	COIL_STOP	HMI_Stop : Commande d'arrêt d'urgence. L'opérateur appuie sur "Stop" pour interrompre immédiatement le cycle en cours.
10	COIL_S_PAINT	HMI_S_Peinture : Commande de déclenchement de la peinture. L'opérateur active cette commande pour lancer la pulvérisation après que la pièce est en position.
11	COIL_S_QC	HMI_S_QC : Commande de contrôle qualité. L'opérateur (ou le système) déclenche la phase d'inspection visuelle de la pièce peinte.
12	COIL_S_OK	HMI_S_OK : Commande de validation qualité. L'opérateur confirme que la pièce est conforme (1 = OK, 0 = Défaut).

Table 2.2 – Table d'adressage Modbus de la ligne de production de peinture

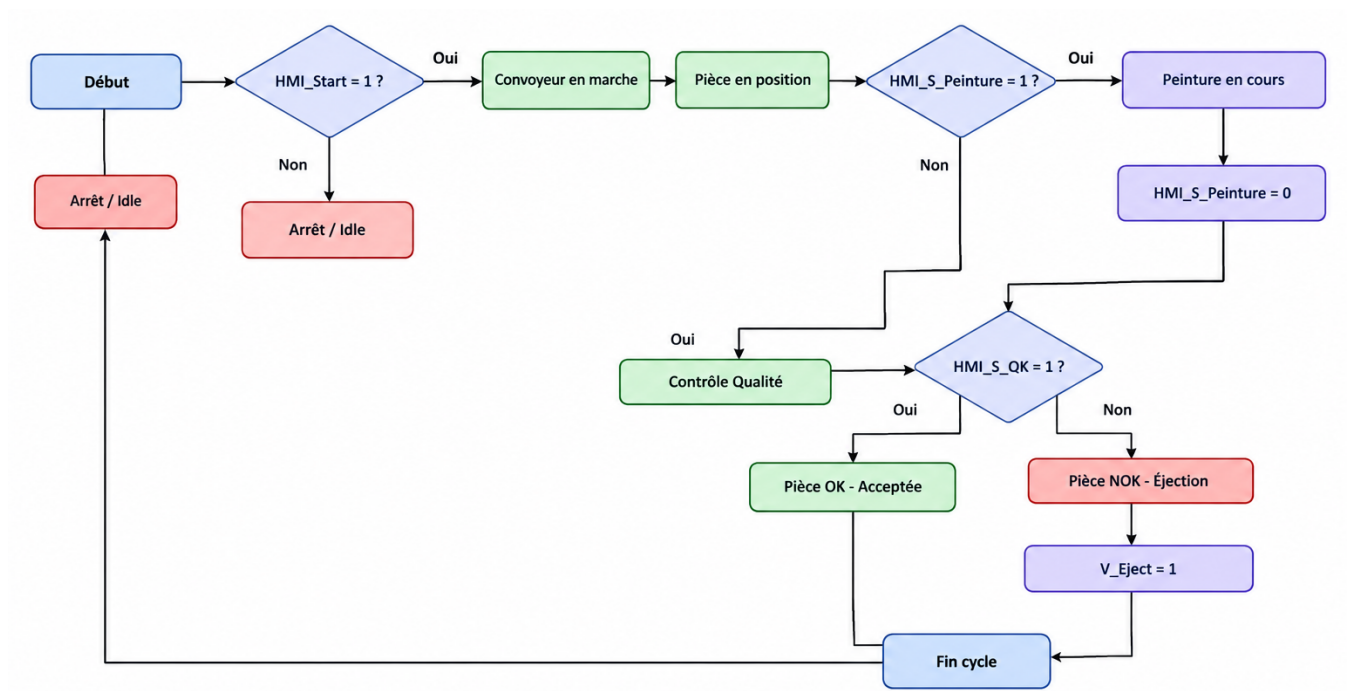


Figure 2.1 – Schema de processus en termes de simulation modbus

ce schema est aussi applicable dans le cas de trafic S7comm.

Voir aussi des exemples dans le generation des variations mathematiques dans les operations industrielles :

```
class ProcessSimulator:
    """Simulateur de processus industriel avec drift continu."""

    def update(self, dt: float, machine_state: int):
        # Mise à jour des variables avec bruit gaussien
        self.pressure = max(0.0, base_p + sine + random.gauss(0, 0.05))
        self.temperature = max(0.0, base_t + sine * 2 + random.gauss(0, 0.3))
        self.speed = max(0.0, base_v + random.gauss(0, 0.03))
```

Listing 2.1 – Extrait de `modbus_server.py` - Serveur et simulateur

La figure 2.2 montre un exemple de capture Wireshark du trafic Modbus TCP généré :

No.	Time	Source	Destination	Protocol	Length	Info
6	4.478519700	127.0.0.1	127.0.0.1	TCP	56	502 → 56151 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM
7	4.478641000	127.0.0.1	127.0.0.1	TCP	44	56151 → 502 [ACK] Seq=1 Ack=1 Win=65280 Len=0
8	4.480897800	127.0.0.1	127.0.0.1	Modbus/T...	56	Query: Trans: 1; Unit: 1, Func: 5: Write Single Coil
9	4.481007700	127.0.0.1	127.0.0.1	TCP	44	502 → 56151 [ACK] Seq=1 Ack=13 Win=65280 Len=0
10	4.482698100	127.0.0.1	127.0.0.1	Modbus/T...	56	Response: Trans: 1; Unit: 1, Func: 5: Write Single Coil
11	4.482780200	127.0.0.1	127.0.0.1	TCP	44	56151 → 502 [ACK] Seq=13 Ack=13 Win=65280 Len=0
12	4.984442800	127.0.0.1	127.0.0.1	Modbus/T...	56	Query: Trans: 2; Unit: 1, Func: 5: Write Single Coil
13	4.984568800	127.0.0.1	127.0.0.1	TCP	44	502 → 56151 [ACK] Seq=13 Ack=25 Win=65280 Len=0
14	4.986035600	127.0.0.1	127.0.0.1	Modbus/T...	56	Response: Trans: 2; Unit: 1, Func: 5: Write Single Coil
15	4.986118400	127.0.0.1	127.0.0.1	TCP	44	56151 → 502 [ACK] Seq=25 Ack=25 Win=65280 Len=0
16	5.689280700	127.0.0.1	127.0.0.1	Modbus/T...	56	Query: Trans: 3; Unit: 1, Func: 5: Write Single Coil
17	5.689528400	127.0.0.1	127.0.0.1	TCP	44	502 → 56151 [ACK] Seq=25 Ack=37 Win=65280 Len=0
18	5.691297700	127.0.0.1	127.0.0.1	Modbus/T...	56	Response: Trans: 3; Unit: 1, Func: 5: Write Single Coil
19	5.691399600	127.0.0.1	127.0.0.1	TCP	44	56151 → 502 [ACK] Seq=37 Ack=37 Win=65280 Len=0
20	6.893540100	127.0.0.1	127.0.0.1	Modbus/T...	56	Query: Trans: 4; Unit: 1, Func: 1: Read Coils
21	6.893640400	127.0.0.1	127.0.0.1	TCP	44	502 → 56151 [ACK] Seq=37 Ack=49 Win=65280 Len=0
22	6.894663600	127.0.0.1	127.0.0.1	Modbus/T...	54	Response: Trans: 4; Unit: 1, Func: 1: Read Coils
23	6.894717500	127.0.0.1	127.0.0.1	TCP	44	56151 → 502 [ACK] Seq=49 Ack=47 Win=65280 Len=0

Figure 2.2 – Capture Wireshark du trafic Modbus TCP (écritures et lectures de coils)

2.2.2 Génération de datasets S7comm classique

Le script `s7comm.py` utilise la bibliothèque `python-snap7` pour simuler un PLC Siemens S7-1200 avec les caractéristiques suivantes :

- Serveur S7comm sur le port 102 (magic byte 0x32)
- Base de données DB1 de 256 octets avec empaquetage Big-Endian
- Simulation physique réaliste de pression, température, vitesse, et RPM (Round per minute)
- Injection d'anomalies probabilistes (0.1% de chances)

Octets	Variable	Type S7	Description / Exemple
— Entrées HMI (Commandes Opérateur) —			
0	Input Coils	BYTE	Bits 0-4 : start, stop, paint_trigger, quality_check, quality_ok
1	(Réservé)	-	Non utilisé
— Sorties PLC (Actionneurs) —			
2	Output Coils	BYTE	Bits 0-2 : conveyor, paint_valve, eject_valve
3	(Réservé)	-	Non utilisé
— Capteurs et Télémétrie —			
4-7	Pressure	REAL	Pression du système (bar). Plage : 0.5 - 6.0 bar
8-11	Temperature	REAL	Température du four (°C). Plage : 20.0 - 60.0 °C
12-15	Conveyor Speed	REAL	Vitesse du convoyeur (m/min). Plage : 0.0 - 2.0 m/min
16-19	Motor RPM	INT	Vitesse de rotation du moteur (RPM). Plage : 0 - 3000 RPM
20-23	Flow Rate	REAL	Débit de fluide (L/min). Plage : 0.0 - 25.0 L/min
24-27	System Uptime	DINT	Temps de fonctionnement (secondes). Incrémentation continue
28-31	Cycle Count	DINT	Nombre de cycles de production complétés
32-35	Paint Temperature	REAL	Température de la peinture (°C). Spécifique à l'état <i>Painting</i>
36-39	Paint Pressure	REAL	Pression de la peinture (bar). Spécifique à l'état <i>Painting</i>
— Réservé pour extensions —			
40-255	Reserved	-	Pour futures télémétries et anomalies

Table 2.3 – Mapping mémoire DB1 du simulateur S7comm (256 octets)

La figure 2.4 montre une capture Wireshark du trafic S7comm généré :

No.	Time	Source	Destination	Protocol	Length	Info
81	6.014691100	127.0.0.1	127.0.0.1	S7COMM	66	ROSCTR:[Ack_Data] Function:[Write Var]
82	6.014775300	127.0.0.1	127.0.0.1	TCP	44	55047 → 102 [ACK] Seq=249 Ack=187 Win=65280 Len=0
83	6.223468000	127.0.0.1	127.0.0.1	S7COMM	75	ROSCTR:[Job] Function:[Read Var]
84	6.223566300	127.0.0.1	127.0.0.1	TCP	44	102 → 55046 [ACK] Seq=187 Ack=280 Win=65024 Len=0
85	6.223677800	127.0.0.1	127.0.0.1	S7COMM	75	ROSCTR:[Job] Function:[Read Var]
86	6.223904700	127.0.0.1	127.0.0.1	TCP	44	102 → 55047 [ACK] Seq=187 Ack=280 Win=65024 Len=0
87	6.224048000	127.0.0.1	127.0.0.1	S7COMM	70	ROSCTR:[Ack_Data] Function:[Read Var]
88	6.224140900	127.0.0.1	127.0.0.1	TCP	44	55046 → 102 [ACK] Seq=280 Ack=213 Win=65280 Len=0
89	6.224276000	127.0.0.1	127.0.0.1	S7COMM	70	ROSCTR:[Ack_Data] Function:[Read Var]
90	6.224351700	127.0.0.1	127.0.0.1	TCP	44	55047 → 102 [ACK] Seq=280 Ack=213 Win=65280 Len=0

Figure 2.4 – Capture Wireshark du trafic S7comm (lecture/écriture de variables)

2.2.3 Génération S7comm Plus (S7+ Sécurisé)

Le script `s7plus.py` implémente une simulation native du protocole S7comm Plus (magic byte 0x72) via des sockets TCP bruts. Cette implémentation est cruciale pour :

- Générer des signatures de trafic distinctes du S7comm classique

```

303 def update_db1(self, inputs: dict, outputs: dict):
304     """Encode process state into DB1 with strict Big-Endian packing."""
305     try:
306         # Bytes 0-1: Input coils (HMI commands)
307         input_byte = 0
308         if inputs.get('start'):
309             input_byte |= (1 << 0)
310         if inputs.get('stop'):
311             input_byte |= (1 << 1)
312         if inputs.get('paint_trigger'):
313             input_byte |= (1 << 2)
314         if inputs.get('quality_check'):
315             input_byte |= (1 << 3)
316         if inputs.get('quality_ok'):
317             input_byte |= (1 << 4)
318         self.db1.data[0] = input_byte

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER

```

=====
2026-09-03 21:51:24 [INFO] S7PLC: PRODUCTION CYCLE #37
2026-09-03 21:51:24 [INFO] S7PLC: =====
2026-09-03 21:51:24 [INFO] S7PLC: [HMI] ▶ Sending START command
2026-09-03 21:51:24 [INFO] S7PLC: [HMI] ✓ START sent
2026-09-03 21:51:25 [INFO] S7PLC: [HMI] ✓ Telemetry: P=1.88bar T=44.9°C S=0.00m/min
2026-09-03 21:51:25 [INFO] S7PLC: [HMI] ▶ Triggering PAINT command
2026-09-03 21:51:25 [INFO] S7PLC: [HMI] ✓ PAINT triggered
2026-09-03 21:51:25 [INFO] S7PLC: [HMI] ▶ Painting in progress (~68s)

```

Figure 2.3 – Demonstration `s7comm.py` en cours d'exécution

- Permettre l'entraînement de modèles ML capables de différencier les deux protocoles
- Simuler des sessions sécurisées avec opérations `CreateObject` et `DeleteObject`

```

class S7PlusPacket:
    """Constructeur et parseur de paquets S7comm Plus."""

    @staticmethod
    def build(opcode: int, payload: bytes) -> bytes:
        # Construction TPKT + COTP + S7plusHeader
        s7plus_header = struct.pack(">BBBBH",
            0x72,          # Protocol ID (magic byte)
            0x01,          # Version
            opcode,        # Opcode (CREATE/DELETE/READ/WRITE)
            0x00,          # Reserved
            len(payload))  # Data length

        cotp = struct.pack(">BBB", 0x02, 0xF0, 0x80)
        total_length = 4 + len(cotp) + len(s7plus_header) + len(payload)
        tpkt = struct.pack(">BBH", 0x03, 0x00, total_length)

        return tpkt + cotp + s7plus_header + payload

```

Listing 2.2 – Extrait de `s7plus.py` - Packet `S7comm Plus`

Contrairement à `s7comm.py` qui utilise la bibliothèque `python-snap7`, ce script adopte une approche de réingénierie du format binaire du protocole `S7comm Plus`. En s'appuyant sur les analyses des échanges réels entre un automate Siemens S7-1200 et une interface HMI TIA Portal, le script construit manuellement chaque trame selon la structure exacte : en-tête `TPKT` (RFC 1006), segment `COTP` (ISO 8073), message `S7comm Plus` avec son magic byte `0x72`, ses numéros de séquence, ses codes de fonction (`CreateObject`, `CreateSubscription`) et son trailer de validation `0x72` en fin de trame.

Cette construction bas niveau, réalisée via des sockets TCP bruts, génère un trafic réseau

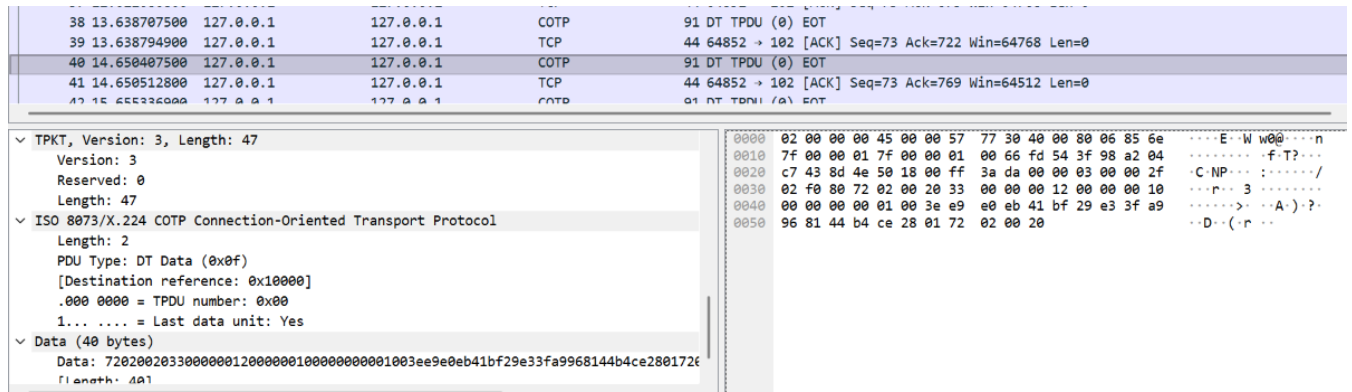


Figure 2.5 – Capture Wireshark du trafic S7comm+

authentique que Wireshark reconnaît et décode nativement avec le plugin S7comm-Plus, sans nécessiter ni matériel Siemens, ni licence TIA Portal, ni implémentation complète de la couche Secure Channel.

Cette approche est essentielles pour l'entraînement de modèles de détection d'intrusions capables de distinguer S7comm classique (0x32) et S7comm Plus (0x72) - le plus pratiquement adapte dans l'industrie - sur la base de leurs seules caractéristiques réseau.

Fonction	Code	Opcode	Description
CreateObject	0x0001	0x22 (Req)/ 0x32 (Resp)	Établissement de session. Le client envoie le type d'objet, le PLC répond avec un handle. <i>Correspond à la connexion initiale TIA Portal.</i>
DeleteObject	0x0002	0x22/0x32	Fermeture de session
CreateSubscription	0x0064	0x22/0x32	Abonnement à des symboles avec recoi d'ID, Envoi d'une liste d'IDs CRC32
DeleteSubscription	0x0065	0x22/0x32	Désabonnement
ModifySubscription	0x0066	0x22/0x32	Modification d'abonnement
TriggerVariableUpdate	0x0012	0x62 (Notif)	Notification asynchrone. Le PLC envoie périodiquement les valeurs des variables souscrites
Read	0x04	0x22/0x32	Accès ponctuel à une variable par son ID de symbole (fallback)
Write	0x05	0x22/0x32	Écriture directe (fallback)

Table 2.4 – Opcodes du protocole S7comm Plus natif

2.3 Prétraitement et Extraction des Données

Le script `ml_processing/notebook1.ipynb` implémente un pipeline complet d'extraction de caractéristiques à partir des captures PCAP et CSV pour la détection d'anomalies dans le réseau prêt à la normalisation.

Extraction des Caractéristiques Protocolaires

Trois parseurs protocolaires sont définis pour traiter les principaux protocoles industriels :

- **ModbusParser** : Extrait les champs MBAP(Transaction ID, Protocol ID, Unit ID, Function Code) ainsi que les métriques PDU(Reference Number, Word Count, Payload Size) port 502.
- **S7CommParser** : Parse S7comm(magic byte 0x32) via les en-têtes TPKT/COTP. Extrait : PDU Type, PDU Reference, Function Code, Subfunction, Area Code, PDU Length, Payload Size.
- **S7CommPlusParser** : Parse S7comm+(magic byte 0x72) avec gestion TPKT/COTP et structures TLV. Extrait : Protocol Version, PDU Type, Sequence Number, Opcode, Function Code(16 bits), Message ID, PDU Length.

La fonction `pcap_to_dataframe()` itère sur les paquets en utilisant `PcapReader` pour un traitement en flux, détecte automatiquement le protocole et construit un `DataFrame pandas` contenant l'ensemble des métriques réseau (IP, TCP, protocole applicatif). Chaque échantillon est étiqueté et les intervalles inter-trames (*Inter-Arrival Time*) sont calculés.

Pipeline de Prétraitement

Le pipeline de prétraitement comprend les étapes suivantes :

1. **Chargement et Nettoyage des Données** : Agrégation des sources PCAP et CSV, suppression des colonnes non-numériques (IP, timestamps, identifiants) et des caractéristiques causant des fuites de données (`frag_offset`, `is_request`, `sUrgRate`, `rUrgRate`, `sFragmentRate`, `rFragmentRate`).
2. **Normalisation** : Application de `StandardScaler` pour centrer-réduire les caractéristiques ($\mu = 0$, $\sigma = 1$) afin de garantir une échelle homogène pour les modèles ML.
3. **Rééquilibrage des Classes** : Détection des classes rares (seuil de 50 échantillons) avec génération de données synthétiques via SMOTE. Pour les classes très rares comme le Port Scan (12 échantillons) et Metasploit (8 samples) , une augmentation spécifique est appliquée avec des variations réalistes (ports scannés, flags TCP, tailles de paquets). Une stratégie SMOTE adaptative définit le nombre de voisins (`k_neighbors=3`) et les échantillons cibles par classe (44,769 échantillons, soit 20% chacun).
4. **Labeling Multi-Classe** : Implémentation d'une architecture multi-classe nécessaire dans les alertes apres incidents pour la différenciation des types d'attaque avec la correspondance suivante :
 - Classe 0 : Normal / Bénin
 - Classe 1 : MITM / PLC Attack
 - Classe 2 : Port Scan / Reconnaissance
 - Classe 3 : Telnet PLC Attack
 - Classe 4 : Web Access Attack
5. **Création des Séquences Temporelles** : Génération de deux formats distincts :
 - **Format 2D** : Matrice [échantillons, caractéristiques] pour les modèles ML (XGBoost,

Random Forest, Isolation Forest).

- **Format 3D** : Séquences temporelles de 10 paquets [échantillons, pas de temps, caractéristiques] pour les modèles (LSTM-Autoencoder). Les séquences sont créées par flux réseau (*flow-based*) en utilisant les adresses IP, ports et protocoles pour préserver la cohérence temporelle.

Le jeu de données final est sauvegardé au format `.npz` avec les variables suivantes :

```
X_train_ML (223845, 73), y_train_ML (223845,),  
X_test_ML (16737, 73), y_test_ML (16737,),  
X_train_DL (223835, 10, 73), y_train_DL (223835,),  
X_test_DL (16727, 10, 73), y_test_DL (16727,).
```

Gestion des Données Déséquilibrées

Dans notre approche, SMOTE a été particulièrement utile pour traiter les classes sous-représentées, notamment la classe **Port Scan / Reconnaissance** provenant de `nmap_cap.pcap`, qui ne contenait initialement que 12 échantillons en raison de l'exclusion des attributs réseau (SYN/ACK, flags TCP) et du faible nombre de paquets utiles. Le pipeline d'augmentation génère 1000 échantillons synthétiques réalistes en variant les ports scannés, les flags TCP et les tailles de paquets, assurant ainsi une représentation équilibrée.

La vérification finale du pipeline inclut la détection des fuites de données (*data leakage*) et l'analyse des caractéristiques constantes par classe, garantissant l'intégrité du jeu de données d'entraînement.

2.4 Entraînement des Modèles

Le notebook `ml_processing/notebook2.ipynb` entraîne les modèles, à partir des données traitées dans le notebook 1 dans `./outputs/processed_dataset.npz`, et évalue trois modèles complémentaires en se basant sur les métriques suivantes :

Isolation Forest (Non-Supervisé)

Modèle d'anomalie basé sur l'isolation aléatoire des échantillons. Entraîné exclusivement sur le trafic normal, il identifie les écarts significatifs comme des attaques.

L'algorithme construit une forêt d'arbres de décision aléatoires. Pour chaque arbre :

1. Un sous-ensemble d'échantillons est sélectionné aléatoirement
2. Une caractéristique est choisie aléatoirement
3. Un point de coupe est sélectionné entre le minimum et le maximum de cette caractéristique
4. Les données sont partitionnées récursivement jusqu'à ce que chaque point soit isolé

```
from sklearn.ensemble import IsolationForest  
  
# Initialisation du modele  
iso_forest = IsolationForest(  
    contamination=0.1, # Proportion attendue d'anomalies (10%)
```

```

    random_state=42,
    n_estimators=100          # 100 arbres dans la forêt
)
iso_forest.fit(X_train_normal)

```

Listing 2.3 – Exemple de détection avec Isolation Forest

XGBoost (Supervisé)

Classifieur par gradient boosting avec les hyperparamètres : `n_estimators=150`, `max_depth=6`, `learning_rate=0.05`. Une analyse d'importance des caractéristiques est générée.

L'algorithme procède de la manière suivante :

1. Un premier arbre est entraîné sur les données
2. Les résidus (erreurs de prédiction) sont calculés
3. Un nouvel arbre est entraîné pour prédire ces résidus
4. Les prédictions sont combinées pondérément
5. Le processus est répété jusqu'à convergence

LSTM-Autoencoder (Deep Learning)

Architecture séquentielle avec encodeur/décodeur LSTM pour la détection d'anomalies par erreur de reconstruction, il reçoit juste le trafic sain et ne nécessite pas d'être adapté à la classification multi-classes.

[!] Binary Vs. Multi-Class architecture

Le code du notebook implémente une architecture hybride **Multi-Task** qui tente de résoudre à la fois la reconstruction (pour la détection d'anomalies) et la classification (pour le type d'attaque). Cependant, comme nous le verrons plus bas, cette approche n'a pas donné de résultats satisfaisants.

```

class MultiTaskLSTMAutoencoder(nn.Module):

    def __init__(self, seq_len, num_features, latent_dim=32, num_classes=5,
                  hidden_dim=128):
        super(MultiTaskLSTMAutoencoder, self).__init__()

        # Encodeur (bidirectionnel pour capter plus de contexte)
        self.encoder_lstm1 = nn.LSTM(num_features, hidden_dim, batch_first=True,
                                     bidirectional=True)
        self.encoder_lstm2 = nn.LSTM(hidden_dim * 2, hidden_dim * 2,
                                     batch_first=True)
        # Projection vers l'espace latent
        self.latent_proj = nn.Linear(hidden_dim * 2, latent_dim)

        # Décodeur
        self.decoder_lstm1 = nn.LSTM(latent_dim, hidden_dim * 2, batch_first=True)
        self.decoder_lstm2 = nn.LSTM(hidden_dim * 2, hidden_dim, batch_first=True)

        # Sortie reconstruite (avec activation Tanh)
        self.decoder_proj = nn.Sequential(nn.Linear(hidden_dim, num_features),
                                           nn.Tanh())

```

```
# Classifieur
self.classifier = nn.Sequential(
    nn.Linear(latent_dim, 256), nn.ReLU(), nn.BatchNorm1d(256),
    nn.Dropout(0.3),
    nn.Linear(256, num_classes)
)
```

Listing 2.4 – Architecture LSTM-Autoencoder (notebook2.ipynb - version réelle)

Le seuil de détection est calibré au 90ème percentile des erreurs de reconstruction du trafic normal (le seuil indiqué dans le rapport initial de 95% ne correspond pas aux paramètres utilisés dans le notebook).

Analyse Comparative

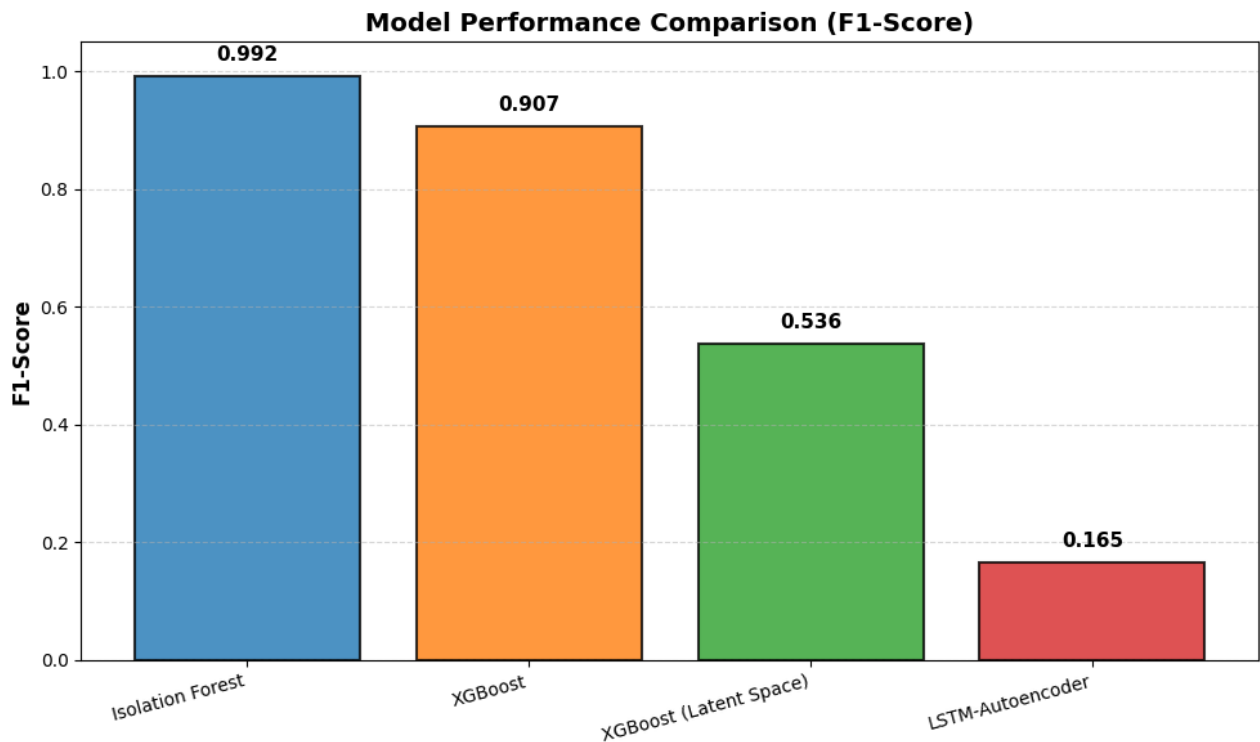


Figure 2.6 – Comparaison des F1-Scores des différents modèles.

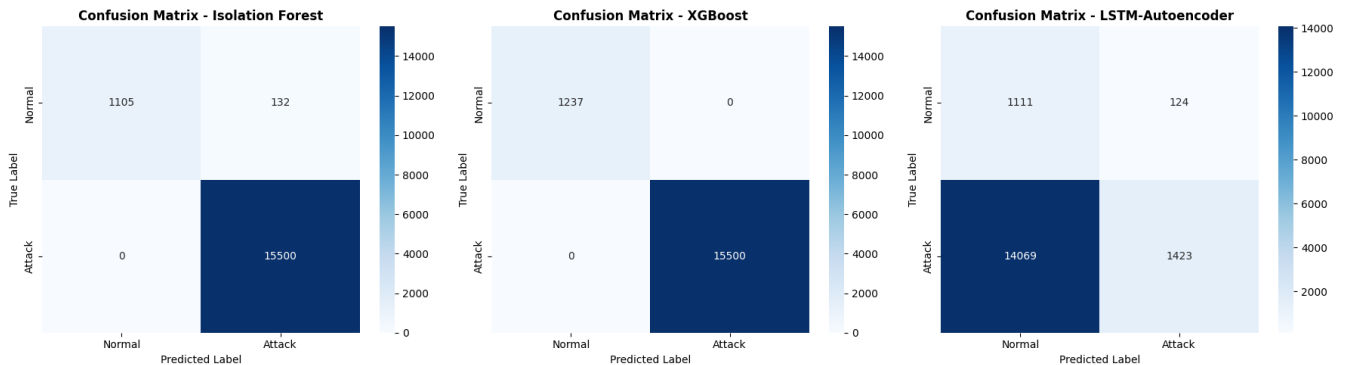


Figure 2.7 – Matrices de confusion pour les modèles Isolation Forest, XGBoost et LSTM-Autoencoder.

Analyse de l'Échec du LSTM-Autoencoder

Comme le montrent les figures 2.6 et 2.7, la performance du LSTM-Autoencoder est significativement inférieure à celle des modèles statistiques (F1-Score de 0.165 contre 0.992 pour Isolation Forest). Plusieurs facteurs expliquent cet échec :

- **Inadéquation aux données tabulaires** : Les LSTM excellent sur des séquences temporelles longues et structurées. Ici, les séquences de 10 paquets sont trop courtes et bruitées pour extraire des motifs pertinents, là où des modèles à base d'arbres sont plus adaptés.
- **Convergence défailante** : La perte de reconstruction stagne à 128 dès les premières époques. L'activation `Tanh` contraint la sortie à $[-1, 1]$, ce qui est incompatible avec la distribution des données normalisées.
- **Déséquilibre des tâches** : Le poids de la classification ($\alpha = 0.7$) domine celui de la reconstruction ($\alpha = 0.3$), poussant le modèle à optimiser la première au détriment de la seconde, rendant le seuil d'anomalie inefficace.
- **Seuil inadapté** : Le seuil au 90ème percentile des erreurs de reconstruction est trop bas ; les distributions d'erreurs du trafic normal et des attaques se chevauchent, rendant la discrimination impossible.
- **Ressources matérielles insuffisantes** : la puissance de calcul ne permet pas de poursuivre successivement les entraînements qui durent plus que de 2 heures

2.5 Évaluation Cross-Dataset

Le notebook `notebook3.ipynb` a pour objectif de répondre à la question cruciale de la robustesse : « **Est-ce que nos modèles fonctionneront sur des données qu'ils n'ont jamais vues ?** ». Il teste les modèles entraînés sur des jeux de données provenant d'environnements différents (matériel, protocoles, réseaux) pour mesurer leur capacité de généralisation réelle.

Objectif

L'objectif principal est de quantifier le phénomène de *domain shift*. Un modèle peut être très performant sur son jeu d'entraînement, mais s'effondrer dès qu'il est confronté à des données provenant d'un autre environnement. Ce notebook mesure cet écart pour déterminer si les modèles sont réellement adaptés à une utilisation en production.

Pipeline d'Évaluation

Le processus se déroule en plusieurs étapes :

1. **Chargement des modèles entraînés** : Les modèles sauvegardés lors du notebook 2 sont rechargés (`isolation_forest.pkl`, `xgboost_model.pkl`, `lstm_autoencoder.pt`, `model_config.json`).
2. **Reconstruction de l'architecture LSTM** : La classe `MultiTaskLSTMAutoencoder` est redéfinie avec les mêmes hyperparamètres que ceux utilisés à l'entraînement.
3. **Chargement des jeux de données externes** : Le notebook tente de charger des données brutes prétraitées (`.npz`) provenant de nouveaux environnements (ex : Siemens S7, OpenPLC Lab, Electra S7Comm).

4. **Évaluation des trois modèles** : Les modèles Isolation Forest, XGBoost et LSTM-Autoencoder sont exécutés sur chaque nouveau jeu de données. Les métriques de précision, rappel, F1-score et exactitude sont calculées pour chaque modèle.
5. **Analyse de l'écart de généralisation** : L'écart (Gap) est calculé comme la différence entre le F1-score maximal et minimal obtenus par un modèle sur l'ensemble des jeux de données. Un écart important indique que le modèle a mémorisé les spécificités de son jeu d'entraînement et ne généralise pas.
6. **Visualisation et rapport** : Les résultats sont compilés dans un tableau (`cross_dataset_evaluation.csv`) et visualisés sous forme de graphiques comparatifs, de heatmaps et de diagrammes d'écart.

Raisons de la Non-Exécution

Ce notebook n'a pas eu la chance d'être exécuté en l'état pour les raisons suivantes :

1. **Absence des fichiers de données externes** : Les fichiers `.npz` anticipés (ex : `siemens_s7_processed.npz`) n'existent pas dans le répertoire de travail. Ils n'ont pas été générés car les datasets correspondants n'ont pas été fournis ou prétraités.
2. **Recours à des données simulées** : En l'absence de ces fichiers, le notebook utilise un bloc de code qui génère des données aléatoires pour simuler le chargement des cross-datasets. Ces données, générées avec une distribution normale aléatoire, sont artificielles et ne représentent pas de manière réaliste un trafic réseau industriel. Elles ne permettent donc pas de tirer des conclusions fiables sur la performance réelle des modèles.
3. **Impossibilité de valider la généralisation** : Sans les vrais datasets externes, il est impossible de mesurer le *generalization gap* et de valider si les modèles sont robustes. L'exécution du notebook avec des données simulées fournirait des métriques trompeuses et non représentatives de la réalité.

2.6 Résultats et Analyse

Performance des Modèles

Le notebook `2-model_trained.ipynb` a entraîné et évalué quatre configurations de modèles. Les performances sont mesurées en termes de précision, de rappel et de F1-score, en se concentrant sur la détection binaire (Normal vs Attaque).

Classe d'Attaque	Précision	Rappel	F1-Score	Support
Normal / Benign	1.000	1.000	1.000	1237
MITM / PLC Attack	0.945	0.913	0.929	11193
Web Access Attack	0.792	0.859	0.824	4287
Telnet PLC Attack	0.818	1.000	0.900	18
Port Scan / Reconnaissance	0.667	1.000	0.800	2

Table 2.5 – Détection par classe d'attaque (modèle XGBoost)

L'Isolation Forest surpasse tous les modèles avec un F1-score de 0.992, démontrant son efficacité sans étiquettes. Le XGBoost atteint 0.907, confirmant la pertinence des modèles à base d'arbres. Les approches par espace latent (0.536) et LSTM seul (0.165) montrent des performances nettement inférieures, soulignant les limites de l'apprentissage profond pour ce problème.

2.6.1 Détection par Protocole

La matrice de confusion du XGBoost multi-classes permet de détailler les performances par type d'attaque.

Classe d'Attaque	Précision	Rappel	F1-Score	Support
Normal / Benign	1.000	1.000	1.000	1237
MITM / PLC Attack	0.945	0.913	0.929	11193
Web Access Attack	0.792	0.859	0.824	4287
Telnet PLC Attack	0.818	1.000	0.900	18
Port Scan / Reconnaissance	0.667	1.000	0.800	2

Table 2.6 – Détection par classe d'attaque (modèle XGBoost)

Le modèle est très performant pour le trafic normal et les attaques majoritaires (MITM, Web Access). Pour "Telnet PLC" et "Port Scan", le rappel est maximal mais la précision est faible, en raison du déséquilibre extrême (seulement 2 et 18 échantillons), rendant l'apprentissage statistiquement difficile.

Analyse de l'Échec du LSTM-Autoencoder

La performance décevante du LSTM ($F1 = 0.165$) s'explique par :

- **Inadéquation aux données** : Les séquences de 10 paquets sont trop courtes et bruitées pour des LSTM, là où les modèles à base d'arbres excellent.
- **Convergence défailante** : La perte de reconstruction stagne à 128 dès les premières époques. L'activation **Tanh** contraint la sortie à $[-1, 1]$, incompatible avec la distribution des données normalisées.

- **Déséquilibre des tâches** : Le poids de classification ($\alpha = 0.7$) domine la reconstruction ($\alpha = 0.3$), rendant le seuil d'anomalie inefficace.
- **Seuil inadapté** : Le seuil au 90ème percentile est trop bas, les distributions d'erreurs se chevauchent, rendant la discrimination impossible.

Analyse du XGBoost sur Espace Latent

Le XGBoost sur espace latent ($F1 = 0.536$) hérite des représentations de mauvaise qualité du LSTM, limitant sa capacité à généraliser.

3 Déploiement et Validation en Environnement Industriel

Ce chapitre présente la validation des modèles d'apprentissage automatique dans un environnement de test quasi-réel, intégrant OpenPLC Runtime, Node-RED et le pipeline de détection temps-réel.

3.1 Architecture de Test

L'environnement de validation reproduit une ligne de production composée de trois couches :

1. **Couche Physique** : OpenPLC Runtime exécutant le programme `.st` de la ligne de peinture
2. **Couche Supervision** : Node-RED assurant l'interface HMI et le contrôle des commandes
3. **Couche Détection** : `sniffer_detector.py` capturant le trafic et exécutant l'inférence ML

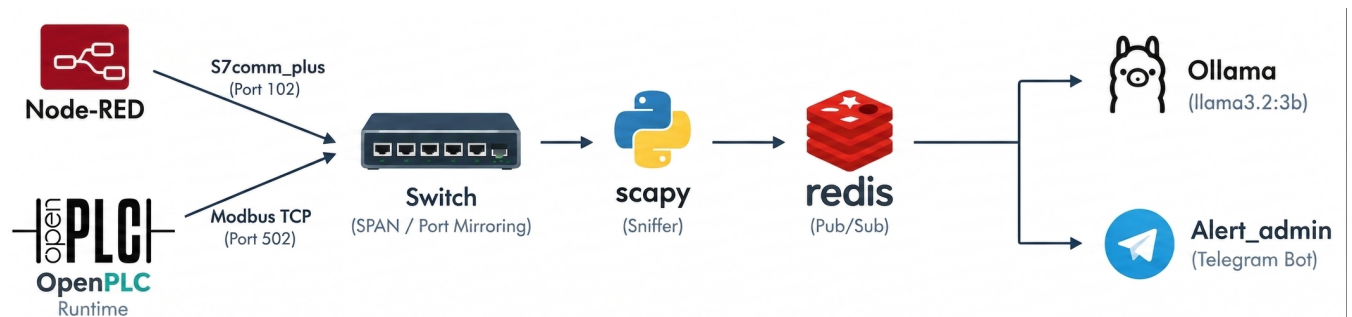


Figure 3.1 – Architecture complète du système de détection en environnement OpenPLC

3.2 OpenPLC Runtime

OpenPLC est utilisé comme un serveur qui exécute le programme de la ligne de production, en agissant avec les commandes qu'il reçoit du serveur scada (Node-red).

Serveurs Actifs

Comme illustré sur la figure 3.2, OpenPLC démarre plusieurs serveurs industriels :

- **Modbus TCP** (port 502) : Communication avec Node-RED et les clients Modbus
- **S7comm** (port 102) : Protocole Siemens via Snap7
- **ENIP** (port 44818) : EtherNet/IP pour la compatibilité avec d'autres équipements

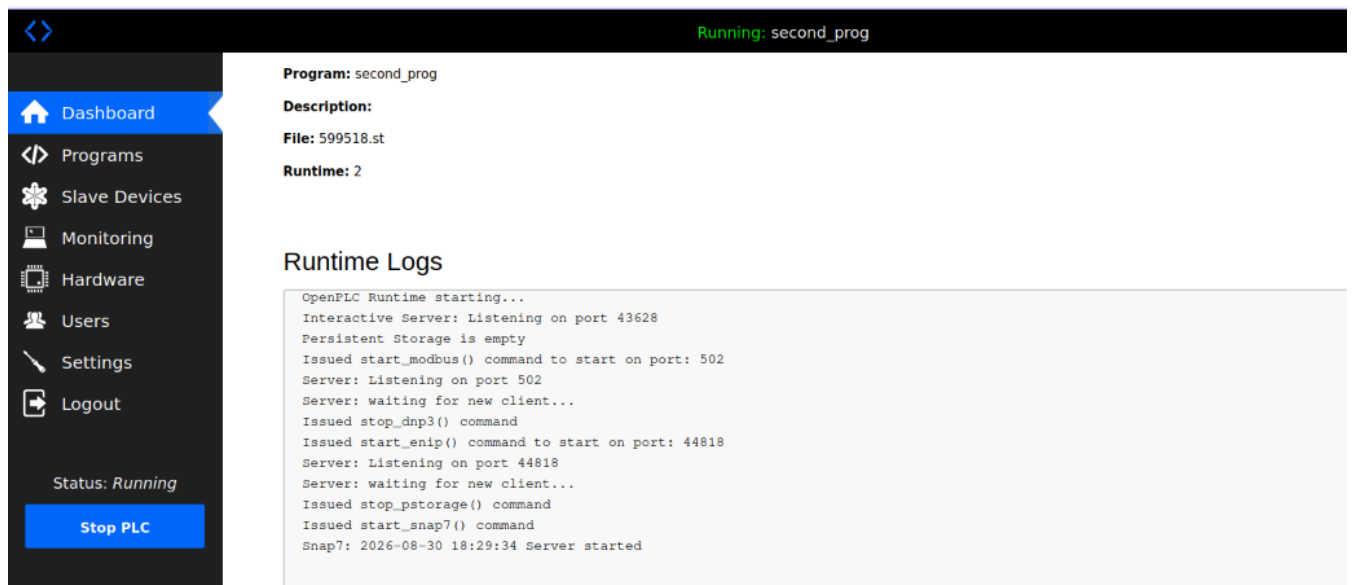


Figure 3.2 – Interface OpenPLC Runtime avec les serveurs actifs

Programme Ladder de la Ligne de Peinture

Ce programme définit une machine à états avec cinq phases : IDLE, RUNNING, PAINTING, QUALITY CONTROL et EJECT. Les temporisations sont configurées comme suit :

- Peinture : 8 secondes
- Contrôle qualité : 5 secondes
- Éjection : 3 secondes

Code source : paint_line.st

Disponible dans le dossier `simulation_scripts/` du projet

3.3 Interface de Supervision Node-RED

Node-RED fournit l'interface de supervision et de contrôle de la ligne de production.

La figure 3.3 montre le graphe Node-RED configuré pour orchestrer le trafic qui vient du program OpenPLC Runtime et de plus le visualiser dans un dashboard bien aussi simple que nécessaire :

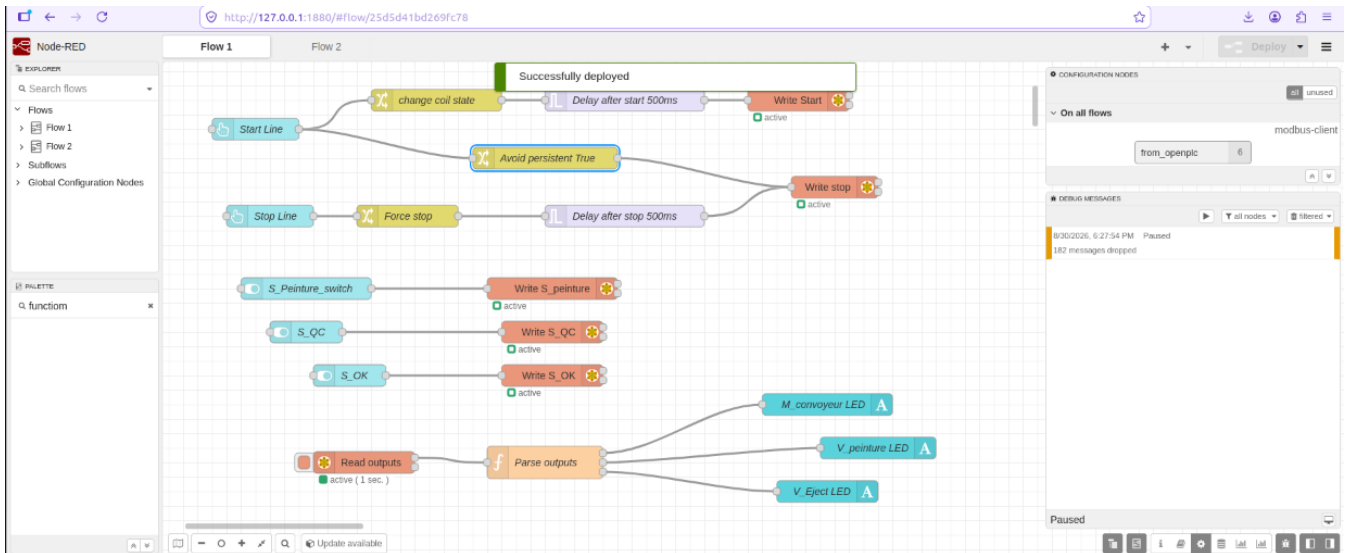


Figure 3.3 – Interface Node-RED pour le contrôle et la supervision

3.3.1 Composants Node-RED

Composant	Fonction
modbus-client	Connexion Modbus TCP vers OpenPLC (port 502)
Write Start/Stop	Écriture des commandes HMI sur les coils Modbus
Delay nodes	Temporisation pour le séquençement des commandes
Read outputs	Lecture périodique des sorties PLC
LED indicators	Affichage visuel de l'état des actionneurs
Switch nodes	Commandes manuelles pour S_Peinture, S_QC, S_OK

Table 3.1 – Composants Node-RED de la supervision

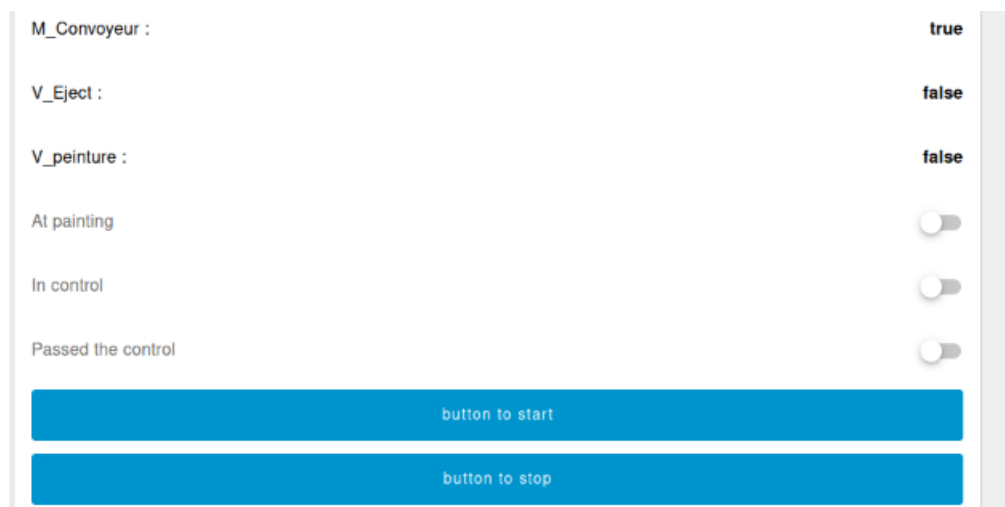


Figure 3.4 – Dashboard interactive base sur le graphe node-red

Scénarios d'Attaque Simulés

Pour implementer les scenarios d'attaques dans le developpement du modele , Quatre types d'attaques ont ete simulés sur ce type de ligne de production pour generer et puis tester son performance, bien a savoir :

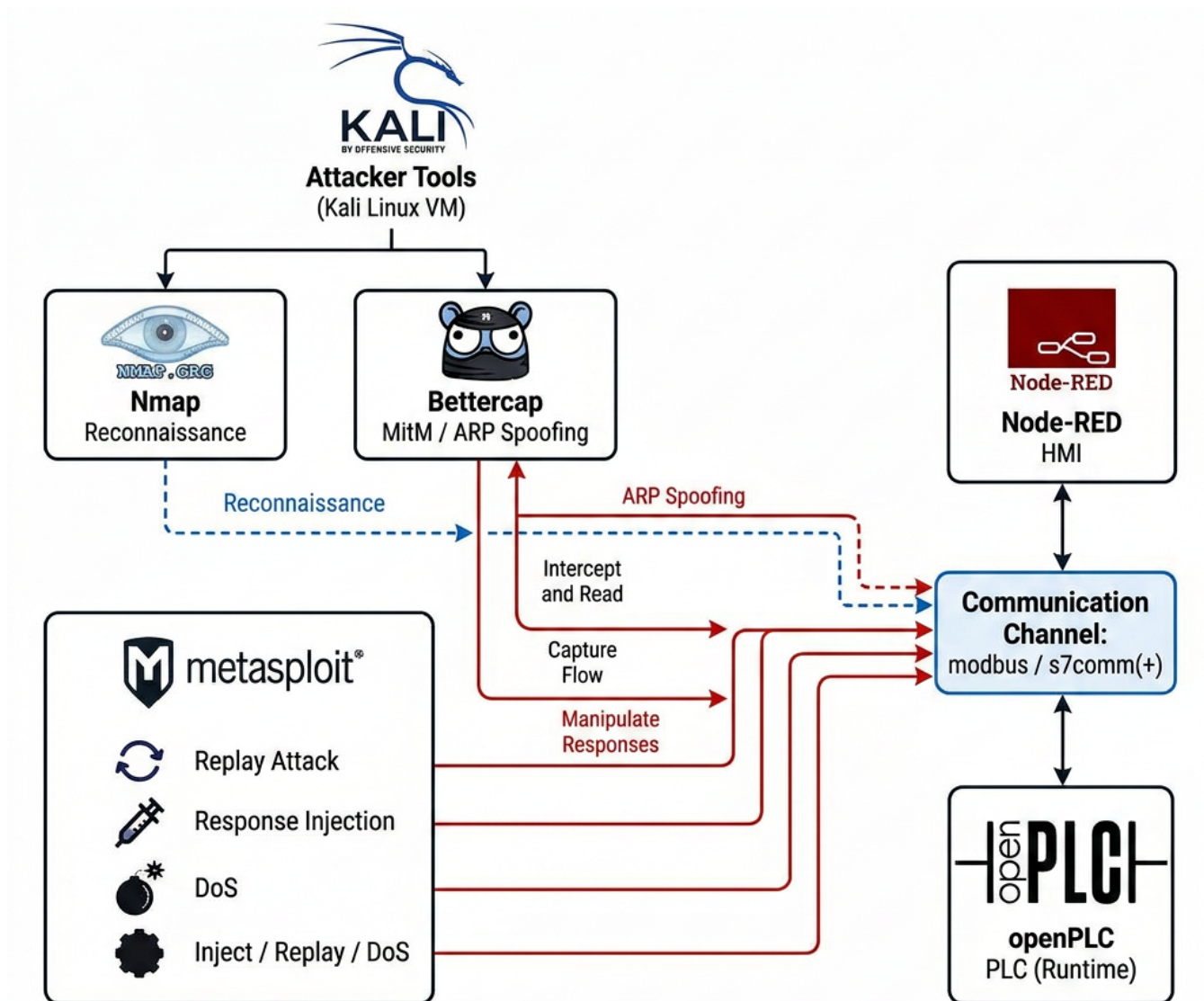


Figure 3.5 – Schema d'attaques ciblant la ligne de communication

- **MITM** : Interception et modification des trames Modbus TCP (désactivation du convoyeur, forçage de l'éjection de chaque pièce ...) , ainsi la connaissance de performance de l'usine et les données sensibles de la part des concurrents
Outils : Bettercap, Ettercap, Scapy, Wireshark (pour capture et réinjection).
- **Replay** : Réinjection de trames capturées (cycles de peinture répétés, activation intempestive du poussoir,)
Outils : Scapy, tcpreplay, Wireshark (capture), Metasploit (auxiliaire Modbus).
- **DoS** : Inondation du réseau Modbus (saturation des communications, arrêt de production)
Outils : hping3, Scapy, Metasploit (auxiliaire de flooding), LOIC, Slowloris.
- **Fuzzing** : Envoi de trames malformées (codes de fonction invalides, données aberrantes)

```
(hamzabk@kali)-[~]
$ sudo bettercap
bettercap v2.41.5 (built for linux amd64 with go1.24.9) [type 'help' for a list of commands]

192.168.153.0/24 > 192.168.153.135 » [18:20:37] [sys.log] [inf] gateway monitor started ...
192.168.153.0/24 > 192.168.153.135 » set arp.spoof.targets 192.168.153.130
192.168.153.0/24 > 192.168.153.135 » arp.spoof on
192.168.153.0/24 > 192.168.153.135 » [18:20:52] [sys.log] [inf] arp.spoof starting net.recon as a requirement for arp.spoof
192.168.153.0/24 > 192.168.153.135 » [18:20:52] [sys.log] [inf] arp.spoof arp spoofer started, probing 1 targets.
192.168.153.0/24 > 192.168.153.135 » [18:20:52] [endpoint.new] endpoint 192.168.153.130 detected as 00:0c:29:fe:1e:ea (VMware, Inc.).
192.168.153.0/24 > 192.168.153.135 » set modbus.inject.function 0x03
192.168.153.0/24 > 192.168.153.135 » set modbus.inject.value 0xDEAD
192.168.153.0/24 > 192.168.153.135 » set modbus.inject.response true
192.168.153.0/24 > 192.168.153.135 » modbus.inject on
192.168.153.0/24 > 192.168.153.135 » [18:21:38] [sys.log] [err] unknown or invalid syntax "modbus.inject on", type help for the help menu
192.168.153.0/24 > 192.168.153.135 » set modbus.inject on
192.168.153.0/24 > 192.168.153.135 » set http.proxy.sslstrip true
192.168.153.0/24 > 192.168.153.135 » set http.proxy.injectjs "alert(modbus injected)"
192.168.153.0/24 > 192.168.153.135 » http.proxy on
192.168.153.0/24 > 192.168.153.135 » [18:22:47] [sys.log] [inf] http.proxy started on 192.168.153.135:8080 (sslstrip enabled)
```

Figure 3.6 – Attaque (MITM) par injection des fausses données en utilisant framework bettercap

```
PORT    STATE SERVICE
102/tcp open  iso-tsap
| s7-info:
| Module: 6ES7 315-2EH14-0AB0
| Basic Hardware: 6ES7 315-2EH14-0AB0
| Version: 3.2.6
| System Name: SNAP7-SERVER
| Module Type: CPU 315-2 PN/DP
| Serial Number: S C-C2UR28922012
| Copyright: Original Siemens Equipment
MAC Address: 00:0C:29:FE:1E:EA (VMware)
Service Info: Device: specialized
```

Figure 3.7 – Scan nmap ciblant modbus en port 502

```
2 auxiliary/scanner/scada/modbus-client - - normal No Modbus client utility
3 \ action: READ_COILS - - - Read bits from several coils
4 \ action: READ_DISCRETE_INPUTS - - - Read bits from several DISCRETE INPUTS
5 \ action: READ_HOLDING_REGISTERS - - - Read words from several HOLDING registers
6 \ action: READ_ID - - - Read device id
7 \ action: READ_INPUT_REGISTERS - - - Read words from several INPUT registers
8 \ action: WRITE_COIL - - - Write one bit to a coil
9 \ action: WRITE_COILS - - - Write bits to several coils
10 \ action: WRITE_REGISTER - - - Write one word to a register
11 \ action: WRITE_REGISTERS - - - Write words to several registers
```

Figure 3.8 – payloads metasploit teste contre l'environnement simule

3.4 Pipeline de Détection en Temps-Réel

Le script `sniffer_detector.py` orchestre l'ensemble du pipeline de détection en deux processus découplés communiquant via Redis Pub/Sub.

Le pipeline fonctionne selon le flux suivant :

1. **Capture** : Sniff des paquets modbus et S7comm via Scapy et port mirroring
2. **Extraction** : Parsing des en-têtes TPKT/COTP et extraction des métriques (pression, température, vitesse)
3. **Publication** : Envoi des données vers Redis Pub/Sub (canal `ics_telemetry_stream`)
4. **Consommation** : Récupération asynchrone des messages par le processus détecteur
5. **Inférence** : Prédiction par le modèle ML chargé (XGBoost ou Isolation Forest)
6. **Alerte** : Notification Telegram en cas d'intrusion et résumé contextuel via Ollama LLM

Fonctions d'Extraction des Caractéristiques

Le script implémente deux parseurs protocolaires :

Fonction	Protocole	Métriques Extraites
<code>extract_s7_features()</code>	S7comm	<code>msg_type</code> , <code>param_len</code> , <code>data_len</code> , pression, température, vitesse
<code>extract_modbus_features()</code>	Modbus TCP	<code>unit_id</code> , <code>func_code</code> , <code>data_len</code> , pression, température, vitesse

Table 3.2 – Fonctions d'extraction des caractéristiques

Points clés de l'extraction Modbus :

- Parsing du MBAP Header (Transaction ID, Protocol ID, Length, Unit ID)
- Vérification du `proto_id = 0` (Modbus TCP)
- Extraction des codes de fonction `0x03` (Read Holding Registers) et `0x04` (Read Input Registers)
- Décodage des valeurs flottantes (32-bit, Big-Endian)

Points clés de l'extraction S7comm :

- Vérification du magic byte `0x32` (S7comm) et `0x72` (s7comm+)
- Parsing du header TPKT (4 octets) et COTP (3 octets)
- Extraction des données DB1 (pression, température, vitesse) en Big-Endian
- Validation des plages physiques (pression 0-5 bar, température 0-50°C)

3.4.1 Processus Producteur (Sniffer Redis)

Le producteur utilise Redis Pub/Sub comme bus de messages asynchrone pour découpler la capture réseau de l'inférence ML. Cette architecture est essentielle car :

- **Le sniffer scapy** capture en continu à haute cadence via filtre BPF (jusqu'à 1000 paquets/s)
- **Le détecteur ML** peut avoir une latence variable (2-10 ms selon le modèle)
- **Redis** agit comme un tampon FIFO, absorbant les pics de trafic

Redis est nécessaire car il offre :

- Une latence < 1 ms, compatible avec les exigences OT
- Un découplage garantissant que le sniffer ne bloque jamais sur l'inférence
- La possibilité d'ajouter plusieurs détecteurs en parallèle (redondance, A/B testing)

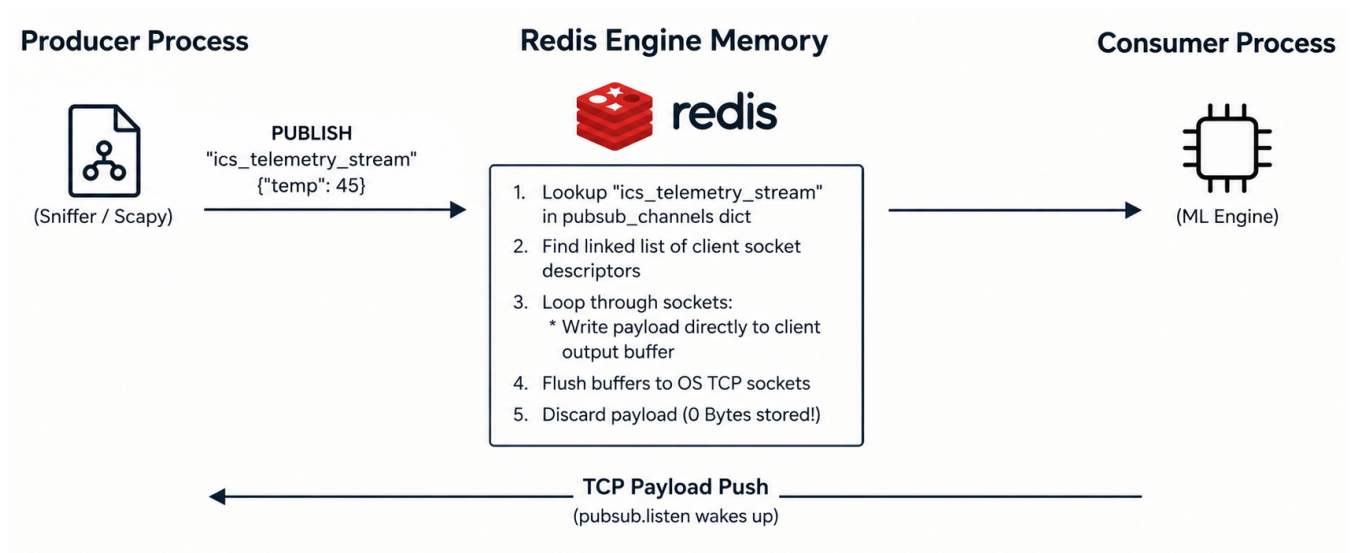


Figure 3.9 – Flux de données Redis entre producteur et consommateur

3.4.2 Processus Consommateur (Détection ML)

Le processus consommateur s'abonne au canal Redis et exécute :

1. Récupération des messages Redis
2. Construction du vecteur de caractéristiques (9 dimensions : protocol ; msg_type ; func_code ; param_len ; data_len ; pression ; temp_four ; vitesse_conv ; has_telemetry)
3. Inférence ML (modèle XGBoost pré-entraîné)
4. Détection d'anomalie (prédiction 1 ou -1)
5. En cas d'anomalie :
 - Appel à l'API Ollama pour résumé contextuel
 - Envoi de l'alerte Telegram

3.4.3 Exemple d'Exécution

```
[CONSUMER] Connected to Redis on localhost:6379
[CONSUMER] No scaler found - using raw features
[!] Error loading models: No module named 'sklearn'
[CONSUMER] Running in heuristic mode (no ML model loaded).
[CONSUMER] Listening for telemetry on Redis channel 'ics_telemetry_stream'...
[CONSUMER] Model type: isolation_forest
[CONSUMER] Écoute commandes Telegram activée
```

Figure 3.10 – Exemple d'exécution du pipeline `sniffer_detector.py`

3.4.4 Performances du Pipeline

Métrique	Valeur	Seuil Critique	Observation
Latence Sniffer → Redis	< 5 ms	10 ms	Capture en temps réel
Latence Redis → Inférence	< 3 ms	5 ms	Consommation asynchrone
Temps d'inférence ML	2-5 ms	10 ms	Modèle XGBoost optimisé
Temps d'alerte Telegram	50-200 ms	500 ms	Dépend du réseau
Latence Ollama (LLM)	2-5 s	N/A	Génération de résumé

Table 3.3 – *Performances du pipeline de détection*

3.4.5 Intégration des Notifications

Génération de Résumés avec Ollama

Lorsqu'une anomalie est détectée, le script interroge l'API Ollama (modèle `llama3.2`) dans l'endpoint `/api/generate` avec un prompt structuré pour générer :

- La nature de l'anomalie (pic de pression, chute de température, type d'attaque)
- L'impact physique sur la ligne de production
- Les recommandations opérationnelles immédiates

Alertes Telegram

Les alertes sont automatiquement envoyées à un bot Telegram (`@ids_icsBot`) avec :

- Un message formaté en HTML
- Les métriques critiques (pression, température, vitesse)
- Le résumé généré par l'agent LLM
- L'IP source de l'anomalie et le type d'attaque si existe avec recommandations

De plus, si l'administrateur veut examiner l'état de sécurité de la chaîne de production dédiée dans les brefs dures, il peut envoyer un message, le bot va consulter le cache en Redis ou également l'historique des messages dans la discussion.

[+] Optimisation des Requêtes Ollama

L'envoi direct de logs d'événements bruts et non filtrés à l'API Ollama sature la fenêtre de contexte du modèle, ralentit la génération et dégrade la qualité des résumés. Pour y remédier, trois stratégies seront raffinées ultérieurement :

- **Fenêtres temporelles fixes** : Filtrage des requêtes sur des intervalles définis (1h, 6h, 24h).
- **Limitation du nombre d'événements** : Restriction de la charge utile JSON à 20-50 événements récents par requête.

- **Résumés agrégés** : Transmission de compteurs synthétiques (ex : *142 pics de pression en 1h*) plutôt que des vidages de paquets bruts.

[!] **Note** : Pour un historique persistant, l'approche recommandée consiste à intégrer une base de données relationnelle (SQLite/PostgreSQL) ou, plus adaptée aux données temporelles, une base de données en série chronologique (InfluxDB/TimescaleDB).

3.5 Conclusion du Chapitre

Ce chapitre a validé le pipeline de détection dans un environnement de test intégrant OpenPLC, Node-RED, Ollama, Redis et Telegram. Toutefois, plusieurs limites sont à souligner :

- **Données** : Rareté des datasets ICS étiquetés, limitant la robustesse des modèles et donc il faut les generer en tout cas.
- **Matériel** : Absence d'équipements physiques, imposant une simulation logicielle imparfaite meme avec des logiciels avancees comme *Factory.io*.
- **Expertise** : Nécessité d'une connaissance de base importante sur des environnements industriels.

Malgré ces contraintes, le pipeline atteint une latence $< 1s$ et confirme la viabilité d'une détection couplant ML et LLM pour la cybersécurité industrielle.

La détection d'intrusions dans les environnements industriels représente un défi critique à l'ère de l'Industrie 4.0. Ce projet démontre la faisabilité d'un système de détection intelligent, temps-réel et multi-protocoles, adapté aux contraintes spécifiques des PLC et des réseaux industriels.

La contribution majeure réside dans la génération de trafic S7comm Plus natif, permettant l'entraînement de modèles capables de distinguer les signatures protocolaires, une capacité essentielle pour détecter les attaques par usurpation de protocole.